HW1

任务1：代码阅读和运行

1.1运行1、2、3代码

1.1.1作业1-鸢尾花分类任务

1. 逻辑回归模型

```
# 打印分类报告
print("分类报告:")
print(classification_report(y_test, y_pred, target_names=['Setosa', 'Versicolor']))
```

```
模型准确率: 1.00
分类报告:
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        12
  Versicolor       1.00      1.00      1.00        18

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

2. 朴素贝叶斯分类器

```
模型准确率: 1.00
分类报告:
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        16
  Versicolor       1.00      1.00      1.00        14

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

3. 支持向量机分类器

```
模型准确率: 1.00
分类报告:
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        17
  Versicolor       1.00      1.00      1.00        13

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30

混淆矩阵:
[[17  0]
 [ 0 13]]
```

1.1.2 作业2-房价预测任务-线性回归模型

```
for feature, coef in zip(boston.feature_names, model.coef_):
    print(f"{feature}: {coef:.2f}")
print(f"截距 (Intercept): {model.intercept_:.2f}")
```

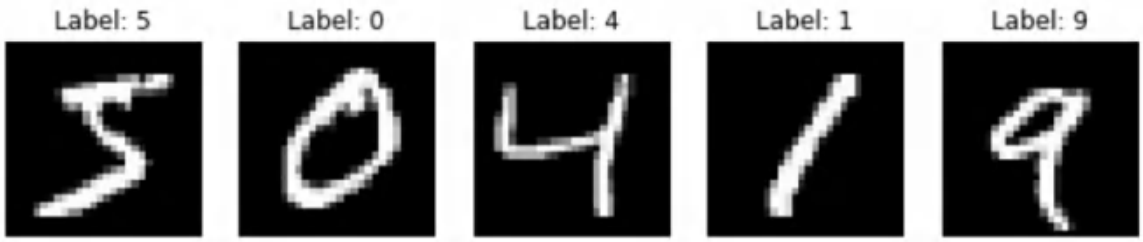
```
均方误差 (MSE): 20.44
决定系数 (R²): 0.67

模型系数:
CRIM: -0.11
ZN: 0.04
INDUS: 0.01
CHAS: 4.24
NOX: -18.51
RM: 4.51
AGE: -0.01
DIS: -1.66
RAD: 0.29
TAX: -0.01
PTRATIO: -0.88
B: 0.01
LSTAT: -0.48
截距 (Intercept): 33.51
```

1.1.3. 作业3-手写数字分类

1. 绘制 MNIST 数据集中前 5个样本图像

```
正在加载 MNIST 数据集...
```



2. 支持向量机分类器

```
正在加载 MNIST 数据集...
训练 SVM 模型...
在测试集上进行预测...
模型准确率: 0.98
分类报告:
              precision    recall  f1-score   support

           0       0.98      0.99      0.98      2072
           1       0.99      0.99      0.99      2346
           2       0.98      0.97      0.97      2113

    accuracy                           0.98      6531
   macro avg       0.98      0.98      0.98      6531
weighted avg       0.98      0.98      0.98      6531
```

3. 逻辑回归分类器

```
正在加载 MNIST 数据集...
训练逻辑回归模型...
在测试集上进行预测...
模型准确率: 1.00
分类报告:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      2139
           1       1.00      1.00      1.00      2295

    accuracy                           1.00      4434
   macro avg       1.00      1.00      1.00      4434
weighted avg       1.00      1.00      1.00      4434
```

1.2.基于决策树的鸢尾花分类代码

1. 代码

```
# 导入必要的库
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report

# 加载鸢尾花数据集
iris = load_iris()
X = iris.data  # 特征
y = iris.target  # 标签

# 只选择 Setosa (0) 和 Versicolor (1) 两类
X = X[y != 2]
y = y[y != 2]

# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# 创建决策树模型
model = DecisionTreeClassifier()

# 训练模型
model.fit(X_train, y_train)

# 在测试集上进行预测
y_pred = model.predict(X_test)

# 计算准确率
accuracy = accuracy_score(y_test, y_pred)
print(f"模型准确率: {accuracy:.2f}")

# 打印分类报告
print("分类报告:")
print(classification_report(y_test, y_pred, target_names=['Setosa',
'Versicolor']))
```

2. 结果

```
print("分类报告:")
print(classification_report(y_test, y_pred, target_names=['Setosa', 'Versicolor']))
```

```
模型准确率: 1.00
分类报告:
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        16
  Versicolor       1.00      1.00      1.00        14

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

任务二：编程实战分析

> 选择作业三中的手写数字分类，分析不同训练条件下分类性能的变化

1.训练样本量

1. 分析内容与思路：分析随着训练样本量的增加，分类器性能（如准确率）的变化趋势。将数据集按比例划分为训练集（5%~40%）和测试集（固定为30%），逐步增加训练样本比例，记录模型在测试集上的准确率。
2. 代码

```python
# 导入必要的库
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# 加载 MNIST 数据集
print("正在加载 MNIST 数据集...")
X, y = fetch_openml('mnist_784', version=1, return_X_y=True,data_home='d:\\')

# 将标签转换为整数
y = y.astype(int)

# 选择 0、1、2三类（结果差距更为明显）
X = X[y < 3]
y = y[y < 3]

#添加数组
```

```
sizes = [0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40]  # 训练集比例
accuracies = []

# 将数据集分为训练集和测试集
for size in sizes:
    # 固定测试集为30%，动态调整训练集比例
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
train_size=size, random_state=42)
    # 创建决策树分类器
    print("训练决策树模型...")
    model = DecisionTreeClassifier(random_state=42)
    # 训练模型
    print("训练决策树模型...")
    model.fit(X_train, y_train)
    # 在测试集上进行预测
    print("在测试集上进行预测...")
    y_pred = model.predict(X_test)
    # 计算准确率
    accuracy = accuracy_score(y_test, y_pred)
    accuracies.append(accuracy)
    print(f"训练集样本量: {size:.2f}")
    print(f"模型准确率: {accuracy:.5f}")
    # 打印分类报告
    print("分类报告:")
    print(classification_report(y_test, y_pred, target_names=['0', '1', '2']))
    cm = confusion_matrix(y_test, y_pred, labels=None, sample_weight=None)
    print(cm)
    print("\n")

plt.figure(figsize=(10,5))
plt.plot(sizes, accuracies, 'bo-')
plt.title("size vs accuracy")
plt.xlabel("train_size")
plt.ylabel("accuracy")
plt.grid(True)
plt.show()
```

3. 运行结果

```
训练集样本量: 0.05
模型准确率: 0.94580
分类报告:
              precision    recall  f1-score   support

           0       0.95      0.96      0.95      2100
           1       0.95      0.97      0.96      2350
           2       0.94      0.90      0.92      2081

    accuracy                           0.95      6531
   macro avg       0.95      0.94      0.94      6531
weighted avg       0.95      0.95      0.95      6531

[[2018   19   63]
 [   7 2285   58]
 [ 102  105 1874]]
训练集样本量: 0.10
模型准确率: 0.95008
分类报告:
              precision    recall  f1-score   support

           0       0.96      0.96      0.96      2100
           1       0.95      0.97      0.96      2350
           2       0.94      0.92      0.93      2081

    accuracy                           0.95      6531
   macro avg       0.95      0.95      0.95      6531
weighted avg       0.95      0.95      0.95      6531

[[2009   24   67]
 [   8 2289   53]
 [  85   89 1907]]
```

训练集样本量: 0.15
模型准确率: 0.95452
分类报告:

	precision	recall	f1-score	support
0	0.95	0.97	0.96	2100
1	0.96	0.98	0.97	2350
2	0.94	0.92	0.93	2081
accuracy			0.95	6531
macro avg	0.95	0.95	0.95	6531
weighted avg	0.95	0.95	0.95	6531

```
[[2028   10   62]
 [   7 2293   50]
 [  89   79 1913]]
```

训练集样本量: 0.20
模型准确率: 0.96417
分类报告:

	precision	recall	f1-score	support
0	0.96	0.97	0.97	2100
1	0.98	0.98	0.98	2350
2	0.95	0.94	0.95	2081
accuracy			0.96	6531
macro avg	0.96	0.96	0.96	6531
weighted avg	0.96	0.96	0.96	6531

```
[[2047    3   50]
 [   5 2297   48]
 [  74   54 1953]]
```

训练集样本量: 0.25
模型准确率: 0.96448
分类报告:

	precision	recall	f1-score	support
0	0.96	0.97	0.97	2100
1	0.97	0.98	0.98	2350
2	0.96	0.93	0.95	2081
accuracy			0.96	6531
macro avg	0.96	0.96	0.96	6531
weighted avg	0.96	0.96	0.96	6531

```
[[2047    7   46]
 [   6 2309   35]
 [  83   55 1943]]
```

在测试集上进行预测...
训练集样本量: 0.30
模型准确率: 0.95942
分类报告:

	precision	recall	f1-score	support
0	0.96	0.97	0.96	2100
1	0.97	0.98	0.97	2350
2	0.95	0.93	0.94	2081
accuracy			0.96	6531
macro avg	0.96	0.96	0.96	6531
weighted avg	0.96	0.96	0.96	6531

```
[[2038    9   53]
 [   6 2294   50]
 [  89   58 1934]]
```

训练决策树模型...
训练决策树模型...
在测试集上进行预测...
训练集样本量: 0.35
模型准确率: 0.96586
分类报告:

	precision	recall	f1-score	support
0	0.96	0.97	0.97	2100
1	0.98	0.97	0.98	2350
2	0.95	0.95	0.95	2081
accuracy			0.97	6531
macro avg	0.97	0.97	0.97	6531
weighted avg	0.97	0.97	0.97	6531

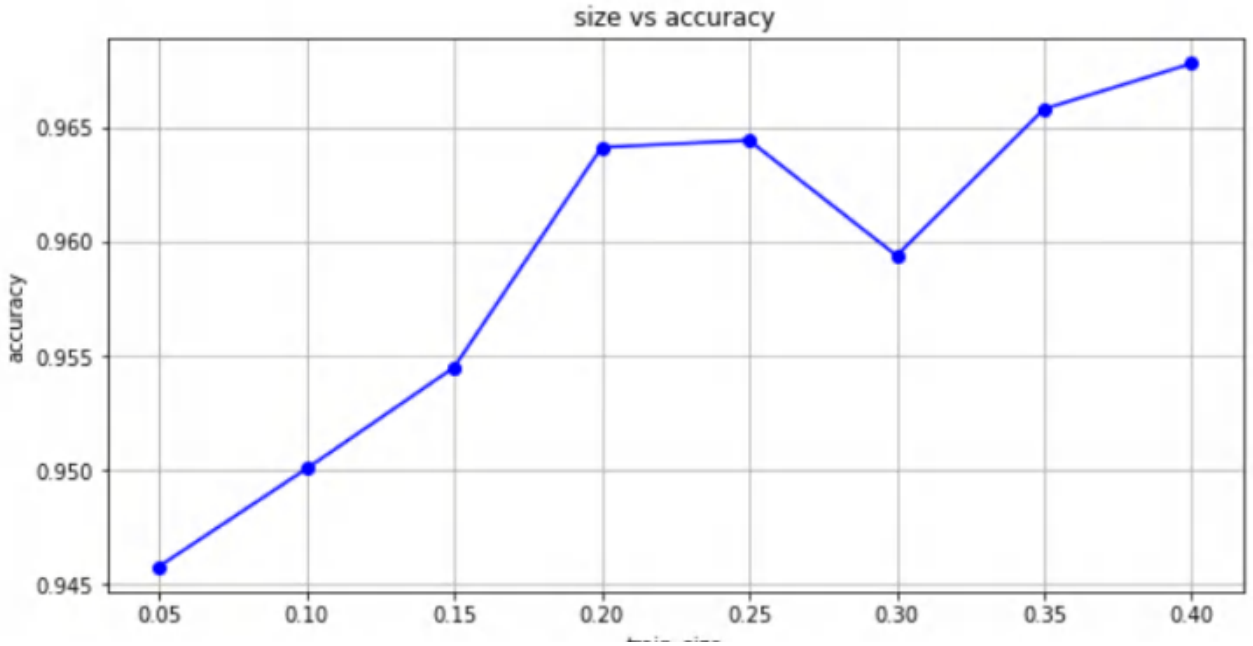
```
[[2046    8   46]
 [   5 2288   57]
 [  72   35 1974]]
```

```
在测试集上进行预测...
训练集样本量: 0.40
模型准确率: 0.96785
分类报告:
              precision    recall  f1-score   support

           0       0.96      0.98      0.97      2100
           1       0.98      0.98      0.98      2350
           2       0.96      0.95      0.95      2081

    accuracy                           0.97      6531
   macro avg       0.97      0.97      0.97      6531
weighted avg       0.97      0.97      0.97      6531

[[2048   10   42]
 [   7 2302   41]
 [  70   40 1971]]
```



4. 结论：决策树分类器随训练样本量的增加而提升，初期（5%~20%）提升显著，后面趋于稳定，又突然的下降可能是因为过拟合导致分类性能下降，充足的训练数据能显著提升模型性能，但边际效益随数据量增加递减。

2.特征维度

1. 分析内容与思路：通过主成分分析（PCA）降低特征维度，观察不同特征数对模型效果的影响。
2. 代码

```
import numpy as np
from sklearn.metrics import confusion_matrix
```

```python
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.decomposition import PCA

# 加载 MNIST 数据集
print("正在加载 MNIST 数据集...")
X, y = fetch_openml('mnist_784', version=1, return_X_y=True,data_home='d:\\')

# 将标签转换为整数
y = y.astype(int)

# 只选择 0-3 3类
X = X[y < 3]
y = y[y < 3]

# 数据归一化
X = X / 255.0

# 设置不同的维度
dimensions = [784, 400, 225, 100, 1]  # 对应 28*28, 20*20, 15*15, 10*10, 1*1
accuracies = []

for dim in dimensions:
    print(f"\n测试维度: {dim}")

    # 使用PCA降维
    if dim < 784:  # 如果需要降维
        pca = PCA(n_components=dim)
        X_reduced = pca.fit_transform(X)
    else:
        X_reduced = X

    # 分割数据集
    X_train, X_test, y_train, y_test = train_test_split(X_reduced, y,
test_size=0.3, train_size=0.2, random_state=42)

    # 训练模型
    model = DecisionTreeClassifier(random_state=42, max_depth=10,
min_samples_split=5)
    model.fit(X_train, y_train)

    # 预测和评估
    y_pred = model.predict(X_test)
```

```
    accuracy = accuracy_score(y_test, y_pred)
    accuracies.append(accuracy)

    # 打印结果
    print(f"维度: {dim}")
    print(f"准确率: {accuracy:.4f}")
    print("分类报告:")
    print(classification_report(y_test, y_pred, target_names=['0', '1', '2']))
    cm = confusion_matrix(y_test, y_pred, labels=None, sample_weight=None)
    print(cm)
    print("\n")

# 绘制结果
plt.figure(figsize=(10,5))
plt.plot(dimensions, accuracies, 'bo-')
plt.title("dimension vs accuracy")
plt.xlabel("dimension")
plt.ylabel("accuracy")
plt.show()


# 绘制不同维度下的重构效果（仅展示第一个样本）
if len(X) > 0:
    plt.figure(figsize=(15, 3))
    plt.subplot(151)
    plt.imshow(X[0].reshape(28, 28), cmap='gray')
    plt.title('Original')

    for i, dim in enumerate(dimensions[1:], 1):  # 跳过原始维度
        pca = PCA(n_components=dim)
        X_reduced = pca.fit_transform(X)
        X_reconstructed = pca.inverse_transform(X_reduced)

        plt.subplot(151 + i)
        plt.imshow(X_reconstructed[0].reshape(28, 28), cmap='gray')
        plt.title(f'Dim={dim}')

    plt.tight_layout()
    plt.show()
```

3. 运行结果

```
测试维度: 784
维度: 784
准确率: 0.9616
分类报告:
              precision    recall  f1-score   support

           0       0.96      0.97      0.97      2100
           1       0.98      0.98      0.98      2350
           2       0.95      0.93      0.94      2081

    accuracy                           0.96      6531
   macro avg       0.96      0.96      0.96      6531
weighted avg       0.96      0.96      0.96      6531

[[2043    7   50]
 [   3 2293   54]
 [  88   49 1944]]



测试维度: 400
维度: 400
准确率: 0.9507
分类报告:
              precision    recall  f1-score   support

           0       0.95      0.96      0.96      2100
           1       0.96      0.98      0.97      2350
           2       0.94      0.91      0.92      2081

    accuracy                           0.95      6531
   macro avg       0.95      0.95      0.95      6531
weighted avg       0.95      0.95      0.95      6531

[[2015    4   81]
 [   6 2294   50]
 [  97   84 1900]]
```

测试维度: 225
维度: 225
准确率: 0.9547
分类报告:

	precision	recall	f1-score	support
0	0.96	0.96	0.96	2100
1	0.97	0.98	0.97	2350
2	0.94	0.92	0.93	2081
accuracy			0.95	6531
macro avg	0.95	0.95	0.95	6531
weighted avg	0.95	0.95	0.95	6531

```
[[2017    3   80]
 [   5 2297   48]
 [  90   70 1921]]
```

测试维度: 100
维度: 100
准确率: 0.9614
分类报告:

	precision	recall	f1-score	support
0	0.96	0.96	0.96	2100
1	0.98	0.98	0.98	2350
2	0.94	0.94	0.94	2081
accuracy			0.96	6531
macro avg	0.96	0.96	0.96	6531
weighted avg	0.96	0.96	0.96	6531

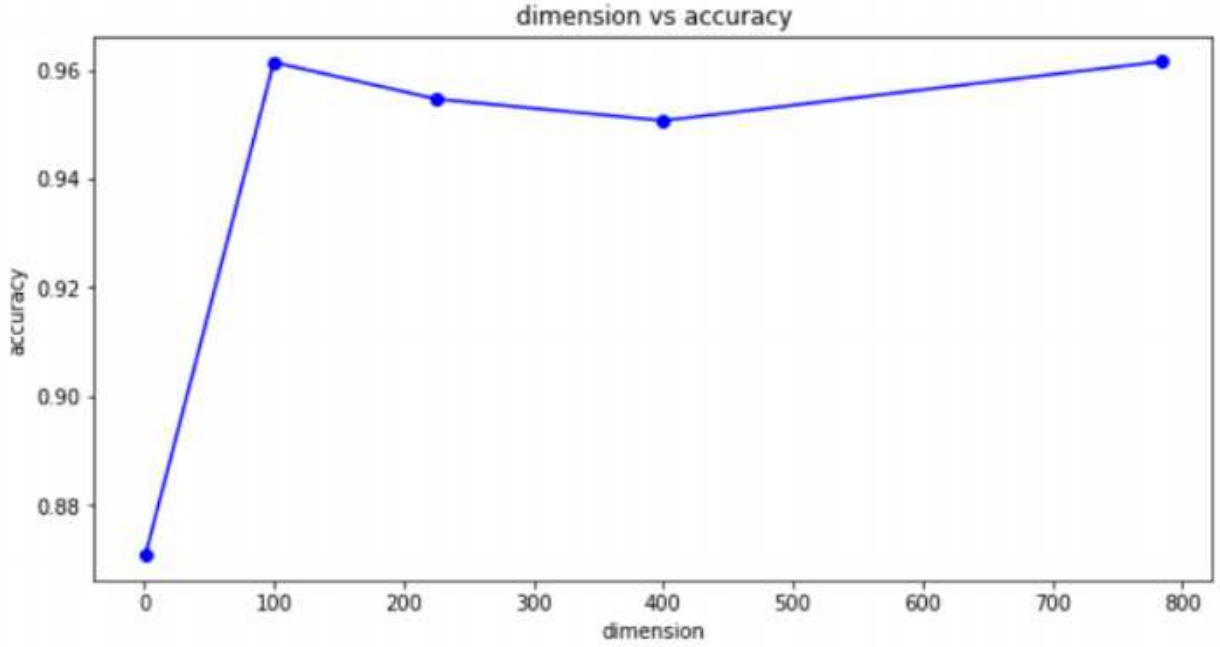
```
[[2023    2   75]
 [   6 2299   45]
 [  76   48 1957]]
```

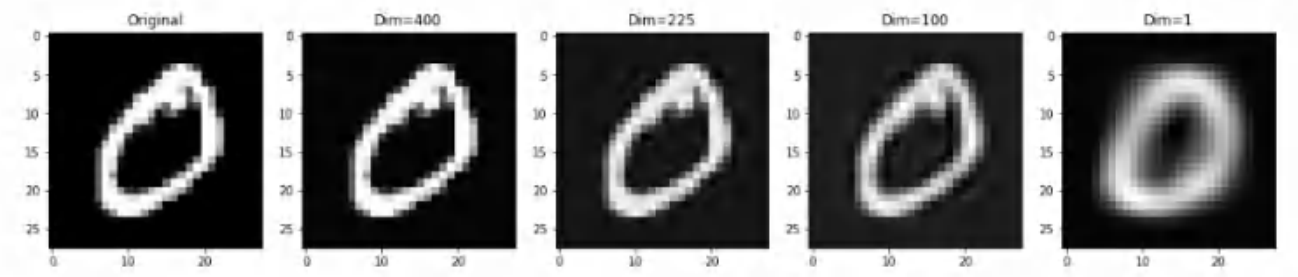
```
测试维度: 1
维度: 1
准确率: 0.8708
分类报告:
              precision    recall  f1-score   support

           0       0.85      0.89      0.87      2100
           1       0.94      0.94      0.94      2350
           2       0.81      0.77      0.79      2081

    accuracy                           0.87      6531
   macro avg       0.87      0.87      0.87      6531
weighted avg       0.87      0.87      0.87      6531

[[1862    1  237]
 [   0 2213  137]
 [ 329  140 1612]]
```





4. 结论：特征维度不足会导致信息丢失，降低分类性能，如10*10降低到1*1维度时，由于信息丢失很多，分类性能显著下降；但过高的维度可能引入冗余，需权衡降维与信息保留。如dim=784, 400, 225，分类性能先下降后上升

3.噪声情况

1. 分析内容与思路：在数据中添加噪声，测试分类器在噪声干扰下的鲁棒性。向训练数据的特征中添加噪声，测试噪声强度与分类准确率的关系。
2. 代码

```
# 导入必要的库
import numpy as np
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# 加载 MNIST 数据集
print("正在加载 MNIST 数据集...")
X, y = fetch_openml('mnist_784', version=1, return_X_y=True,data_home='d:\\')

# 将标签转换为整数
y = y.astype(int)

# 只选择 0 和 1 两类
X = X[y < 3]
y = y[y < 3]

# 数据归一化
X = X / 255.0

# 设置不同的噪声级别和训练集大小
noise_levels = [0.1, 0.3, 0.5, 0.7, 0.9]  # 归一化后的噪声级别
accuracies = []

for noise in noise_levels:
    # 给图片添加随机噪声
    noise_matrix = np.random.uniform(-noise, noise, size=X.shape)
    noisy_X = X + noise_matrix
    noisy_X = np.clip(noisy_X, 0, 1)  # 确保值在[0,1]范围内

    # 分割数据集
    X_train, X_test, y_train, y_test = train_test_split(noisy_X, y,
test_size=0.3, train_size=0.05, random_state=42)

    # 训练模型
    print("训练决策树模型...")
    model = DecisionTreeClassifier(random_state=42, max_depth=10,
min_samples_split=5)
```

```
    model.fit(X_train, y_train)

    # 在测试集上进行预测
    print("在测试集上进行预测...")
    y_pred = model.predict(X_test)

    # 记录结果
    accuracy = accuracy_score(y_test, y_pred)
    accuracies.append(accuracy)
    print(f"噪声情况: {noise}")
    print(f"模型准确率: {accuracy:.5f}")
    # 打印分类报告
    print("分类报告:")
    print(classification_report(y_test, y_pred, target_names=['0', '1', '2']))
    cm = confusion_matrix(y_test, y_pred, labels=None, sample_weight=None)
    print(cm)
    print("\n")

# 绘制结果
plt.figure(figsize=(10,5))
plt.plot(noise_levels, accuracies, 'bo-')
plt.title("noise_level vs accuracy")
plt.xlabel("noise_level")
plt.ylabel("accuracy")
plt.show()
```

3. 运行结果

```
在测试集上进行预测...
噪声情况: 0.1
模型准确率: 0.91548
分类报告:
              precision    recall  f1-score   support

           0       0.93      0.93      0.93      2100
           1       0.93      0.95      0.94      2350
           2       0.89      0.86      0.87      2081

    accuracy                           0.92      6531
   macro avg       0.91      0.91      0.91      6531
weighted avg       0.92      0.92      0.92      6531

[[1954   21  125]
 [  12 2236  102]
 [ 146  146 1789]]


训练决策树模型...
在测试集上进行预测...
噪声情况: 0.3
模型准确率: 0.91732
分类报告:
              precision    recall  f1-score   support

           0       0.92      0.94      0.93      2100
           1       0.94      0.94      0.94      2350
           2       0.89      0.87      0.88      2081

    accuracy                           0.92      6531
   macro avg       0.92      0.92      0.92      6531
weighted avg       0.92      0.92      0.92      6531

[[1983   22   95]
 [   7 2204  139]
 [ 162  115 1804]]
```

噪声情况: 0.5
模型准确率: 0.88164
分类报告:

	precision	recall	f1-score	support
0	0.92	0.91	0.91	2100
1	0.89	0.92	0.90	2350
2	0.84	0.82	0.83	2081
accuracy			0.88	6531
macro avg	0.88	0.88	0.88	6531
weighted avg	0.88	0.88	0.88	6531

```
[[1904   45  151]
 [  15 2155  180]
 [ 161  221 1699]]
```

训练决策树模型...
在测试集上进行预测...
噪声情况: 0.7
模型准确率: 0.80478
分类报告:

	precision	recall	f1-score	support
0	0.84	0.82	0.83	2100
1	0.84	0.84	0.84	2350
2	0.73	0.74	0.74	2081
accuracy			0.80	6531
macro avg	0.80	0.80	0.80	6531
weighted avg	0.81	0.80	0.81	6531

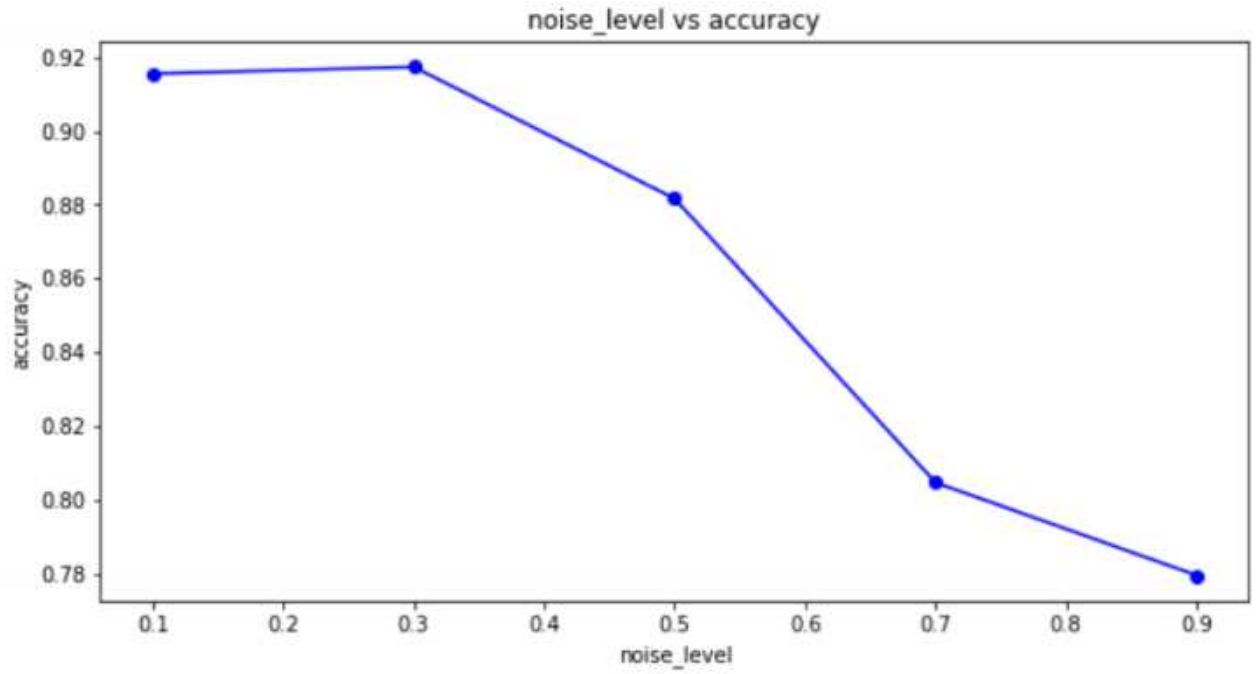
```
[[1728  107  265]
 [  61 1979  310]
 [ 260  272 1549]]
```

```
在测试集上进行预测...
噪声情况：0.9
模型准确率：0.77967
分类报告：
              precision    recall  f1-score   support

           0       0.79      0.78      0.78      2100
           1       0.83      0.88      0.86      2350
           2       0.70      0.67      0.69      2081

    accuracy                           0.78      6531
   macro avg       0.78      0.78      0.77      6531
weighted avg       0.78      0.78      0.78      6531

[[1634  107  359]
 [  56 2065  229]
 [ 380  308 1393]]
```



4. 结论：噪声对分类性能有显著负面影响，随着噪声强度增加，分类性能下降显著，特别是在0.5~0.7，噪声强度过大损失重要信息使得分类性能显著下降

3.结论：

1. **训练样本量**是模型性能的基础，对于本数据集，建议至少覆盖数据分布的20%以上。
2. **特征维度**需根据数据特性选择，保留关键特征可提升模型效率与泛化能力，对于本数据集至少保证10*10维度
3. **噪声**对分类性能负面影响较大，在处理数据时尽量减小噪声以降低对分类器性能的影响